

Hostel Management System

BI INDIVIDUAL PROJECT
RAJ, CHARMY PRASESH

Table of Contents

Introduction	1
Business Problem	2
Justification	2
Purpose	2
Resulted Outcome:	3
Facts and Measures	3
Dimensions:	4
Process (Step by Step Guide):	6
Cube 1: Fee Payment Cube	6
Cube 2: Room Allocation Cube	18
Lessons Learned/3Ws:	28
a. What went well:	28
b. What did not go well:	29
c. What would you do differently next time:	30
Conclusion:	31

Introduction

Managing hostel operations is never just about allocating rooms or collecting fees it's about making sure everything runs smoothly, transparently, and with minimal manual effort. But in most educational institutes, including the context I designed this for, the data is scattered room allocations are handled in one place, fee records in another, and student demographic data somewhere else entirely. When I looked at how hard it was to get answers to simple questions like who hasn't paid their fees or how many rooms are still vacant, I realized how much potential there was for improvement through better data management.

That's what inspired me to take on this project. I wanted to move away from fragmented, spreadsheet-based data tracking and instead create something more structured and analytical, a proper Business Intelligence (BI) system using **SQL Server, Star Schema design, and SSAS cubes**. With this system, even someone without technical skills could gain clear, actionable insights with just a few clicks.

Business Problem

The main problem I tackled was the lack of a centralized, analytical view of hostel data. Everything was in operational form, meaning the data was stored to just *record* transactions, not to *analyze* them. This made it very difficult to:

- Identify students with unpaid fees
- Monitor room occupancy and availability efficiently
- Analyze trends over time (e.g., payment behavior by month or location)
- Group or filter data by useful dimensions like gender, hometown, or room type

Moreover, when attempts were made to pull this information manually, it involved a lot of repeated steps, human error, and time. As someone who believes in automation and clarity, I saw this as an opportunity to build something that could help solve those issues.

Justification

A well-designed **data warehouse** and **OLAP cube** can make a massive difference. Instead of trying to piece together information every time, we can pre-structure everything using **star schemas** which are simple, flat models that are easy to query and visualize.

Using **SSAS cubes**, I could let the system handle aggregations, comparisons, and filters instantly. Users don't have to worry about writing SQL or understanding complex joins. They can just use PivotTables or connect to a dashboard and interact with the data directly.

Also, once this foundation is in place, it's very easy to scale. We could add cubes for **mess billing**, **maintenance requests**, **student complaints**, or even **attendance monitoring** all within the same framework. It's a scalable, sustainable solution that goes way beyond a one-time analysis.

Purpose

My main aim was to show that **data can be transformed into decision-ready information** even in environments that aren't traditionally considered "data-driven." Through this project, I wanted to:

- **Clean and restructure existing data** into star schemas that are optimized for analysis
- Build **two separate cubes**, one for **Fee Payments** and another for **Room Allocations**
- Connect the cubes to **Excel** for interactive analysis using PivotTables
- Derive insights like fee defaulter patterns, room occupancy by student demographics, and trends over time
- Lay the foundation for more BI modules in the future

By the end of the project, not only did I have working cubes and dashboards, but I also had a better appreciation for the power of thoughtful data modeling and how it can simplify real-world operations like hostel management.

Resulted Outcome:

The outcome of this project was both technically sound and practically meaningful. I was able to transform scattered hostel data into an organized, query-friendly structure using **Business Intelligence (BI)** techniques. The use of **SSAS cubes** allowed me to analyze data in multiple ways, which would have otherwise taken hours using traditional spreadsheets or manual filtering.

Key outcomes include:

- I created two fully functional **data marts**, one for **Fee Payments** and another for **Room Allocation**. These data marts were structured using the **Star Schema model**, which made it easy to manage and retrieve data.
- I designed and implemented **Fact Tables** to store measurable business events like fee payments and room allotments, and **Dimension Tables** to provide context like student info, gender, location, and time.
- I successfully deployed both cubes in **SQL Server Analysis Services (SSAS)** without any processing errors, which indicates a well-modeled schema and reliable ETL process.
- Using **Excel PivotTables**, I was able to easily explore and generate valuable insights, such as:
 - Which students have not paid their fees
 - What months see the highest number of payments
 - Which room types are most in demand
 - Which cities have higher hostel occupancy
- My own understanding of BI concepts like **ETL (Extract, Transform, Load)**, **multidimensional data modeling**, and **hierarchies** improved significantly throughout the project.

Facts and Measures

Fact Table 1: Fact_FeePayments

This fact table captures the fee transactions of students. It stores numeric data that can be analyzed to answer questions like "How much has each student paid?" or "Which month had the highest collections?"

Column	Description
PaymentID	A unique identifier for each payment made.
StudentID	Foreign key referencing student information (from Dim_Student).
FSID	Fee Status ID (e.g., Paid or Pending) linked to Dim_FeeStatus.
PaymentAmount	The main measure – total money paid.
PaymentDate	Linked to Dim_Date for time-based filtering (month, year, etc.).

Key Measures:

- **Total Payment Amount** – to see how much revenue has been collected.
- **Average Payment** – to compare payment behavior across students or groups.
- **Count of Payments** – to track how many transactions were processed.

Fact Table 2: Fact_RoomAllocations

This fact table stores information about room allotments to students in the hostel.

Column	Description
AllotmentID	Unique ID for each room allocation event.
StudentID	Linked to Dim_Student_Room, containing student details.
RoomID	Linked to Dim_Room, identifying the room assigned.
DateAllotted	Linked to Dim_DateRoomAllot to track when the room was allocated.

Key Measures:

- **Room Allocation Count** – total number of rooms assigned.
- **Room Capacity Used** – helps in tracking hostel occupancy.
- **Vacancy Rate** – indirectly calculated to monitor how many rooms are unassigned.

Dimensions:

Dimension Tables provide the **context** or **attributes** that describe the facts. They allow the data in the fact tables to be **grouped, filtered, and categorized**. For example, instead of just knowing how much was paid, you can know *who paid it, from which city, and in which month*.

Cube 1: Fee Payment Data Mart

This cube is designed to analyze **student fee payment behavior**, patterns, and trends.

Fact Table: Fact_FeePayments

Holds measurable financial data (e.g., amount paid).

Dimensions

Dimension Table	Description
Dim_Student	Student information such as name, age, gender ID (GID), and location ID (AID).
Dim_Gender	Gender types with values like Male, Female, Other.
Dim_Location	Hometown of students – includes Address, City, State, Country.
Dim_FeeStatus	Whether the fee was Paid or Pending.
Dim_Date	Extracted from PaymentDate. Includes day, month, year, quarter, etc.

Cube 2: Room Allocation & Occupancy Data Mart

This cube supports **hostel management**, focusing on student room allotments and occupancy tracking.

Fact Table: Fact_RoomAllocations

Holds allocation records like which student got which room and when.

Dimensions

Dimension Table	Description
Dim_Student_Room	Student details similar to Dim_Student, but specifically for room data.
Dim_Gender	Gender classification.
Dim_Location	Hometown or permanent residence of the student.
Dim_Room	Room details: RoomType (e.g., Single/Shared), Capacity, Status ID.
Dim_RoomStatus	Indicates whether room is Occupied or Vacant.
Dim_DateRoomAllot	Extracted from DateAllotted. Includes day, month, year, etc.

Process (Step by Step Guide):

Cube 1: Fee Payment Cube

The project began with data preparation in Microsoft Access, where various hostel-related tables such as Student, Fee, Gender, Address, and FeeStatus were organized and linked through appropriate key foreign relationships. To ensure seamless integration with SQL Server Analysis Services (SSAS), the database was saved in the older .mdb format, which offered better compatibility and fewer connection issues during the data source setup.

Once the foundational schema was in place, I created structured queries to build the dimension and fact tables needed for the star schema of the fee payment cube.

The following dimension queries were designed:

- dimStudent: Included StudentID, First Name, Last Name, Age, Contact Number, Gender ID (GID), and Address ID (AID). This served as the core reference for identifying and categorizing students.
- dimGender: A simple dimension mapping Gender ID (GID) to Gender (e.g., Male, Female, Other), enabling demographic filtering.
- dimLocation: Derived from the Address table, it included Address ID (AID), City, State, and Country to analyze fee data across different regions.
- dimFeeStatus: This dimension included the Fee Status ID (FSID) and Situation (Paid or Pending), allowing classification of student payments.
- dimDate: Unlike the other dimensions, this was created manually in Excel to ensure it had a full calendar of dates. It was then imported into the database as a table and included attributes like DateKey, Day, Month, Quarter, and Year. This allowed for consistent and reliable time-based analysis, regardless of missing or null values in the original Fee table.

The central fact table, factFeePayments, was built using a query that joined the Fee table with all necessary foreign keys and included key measures such as Payment Amount and Payment Date. This table was essential for tracking payments, analyzing revenue, and identifying unpaid dues.

After all queries were finalized and verified, I imported the data into SQL Server and deployed the cube using the SSAS Multidimensional Model. Finally, I used Excel PivotTables to slice and filter the data across dimensions such as gender, location, and time, generating actionable insights like:

- Monthly trends in fee collection
- Regional fee payment performance
- Student segments with pending payments

This cube formed the foundation for dynamic reporting and future expansion into a full-fledged hostel analytics system.

Dim_Student
StudentID
FName
Lname
Age

Fig 1.1: Dimension Student Query Field

Dim_Location
AddressID
City
State
Country

Fig 1.2: Dimension Location Query Field

Dim_Gender
GID
Gender

Fig 1.3: Dimension Gender Query Field

Dim_FeeStatus
FSID
Situation

Dim_FeePayment
Situation
FeeID

Fig 1.4: Dimension FeeStatus and FeePayment Query Field

Dim_Date
DateValue
Day
Month
Quarter
Year
PaymentID

Fig 1.5: Dimension Date Query Field

To handle the time dimension, a separate PaymentDate table was created in Excel and later imported into the database. This table was then connected to the fact table query using the PaymentDate attribute from the fee records. This approach ensured that the time dimension was properly linked to the relevant payment data without causing cross-cartesian product issues or null-date inconsistencies.

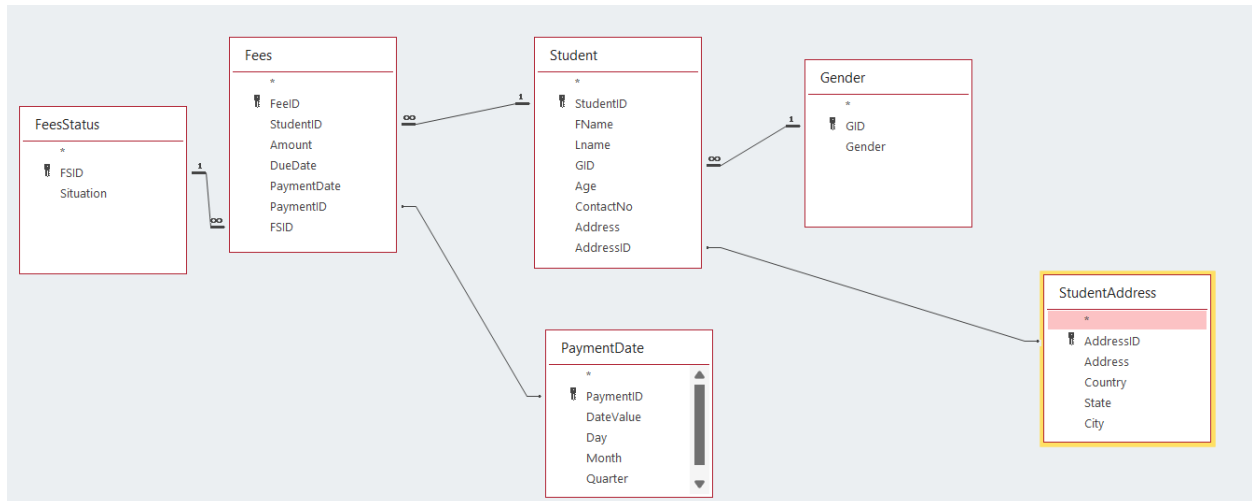


Fig 2.0: Create Time relationship in Fact query.

The fact table query was carefully designed to include all the necessary foreign keys (StudentID, GID, AID, FSID, and PaymentDate) along with the measure: AmountPaid. These keys allowed seamless linking with the associated dimension tables; Student, Gender, Location, FeeStatus, and Date.

The measure AmountPaid was chosen to directly reflect fee collection, supporting the main objective of the project: tracking student payments, identifying pending fees, and generating time-based and region-wise fee reports. By integrating these attributes and measures into the star schema, the project enabled dynamic, drill-down reporting through SSAS and Excel PivotTables.

This cube ultimately helps the hostel administration enhance financial transparency, monitor payment trends, and make data-driven decisions about student fee management and planning.

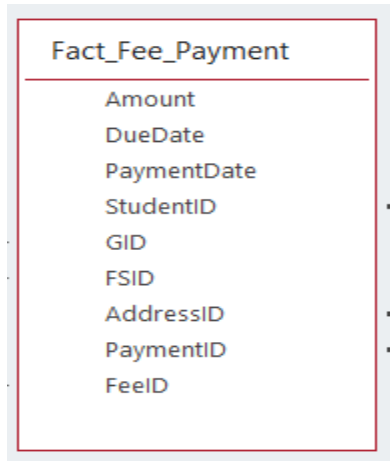


Fig 3.0: Fact Query Field

After creating the dimension and fact table queries, relationships were established between the tables using the Relationships tab in Microsoft Access. The dimStudent, dimGender, dimFeeStatus, dimLocation, and dimDate tables were all connected to the factFeePayments table using their respective foreign keys ensuring referential integrity and enabling accurate analytical processing.

These relationships ensured that each student's payment could be traced across various dimensions like gender, hometown, fee status, and time of payment. This structured connection not only supported clean data extraction into SSAS but also ensured that Excel PivotTables produced meaningful, sliceable, and drillable insights without data redundancy or inconsistency.

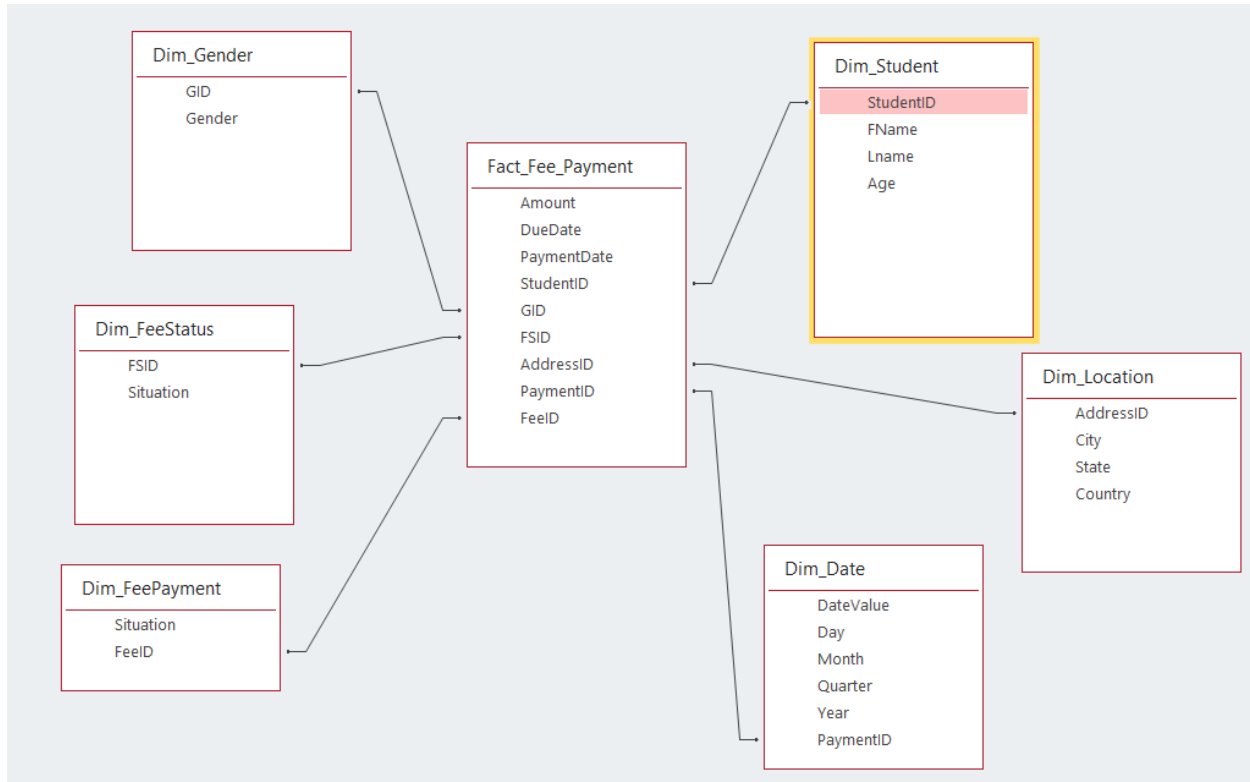


Fig 4.0: Established Relationship with Fact Table

With the Access database prepared, the next step involved uploading the data into Microsoft SQL Server 2022. To accomplish this, the SQL Server Configuration Manager was used to start the necessary services, including SQL Server Integration Services, SQL Server Analysis Services, SQL Server, and SQL Server Browser.

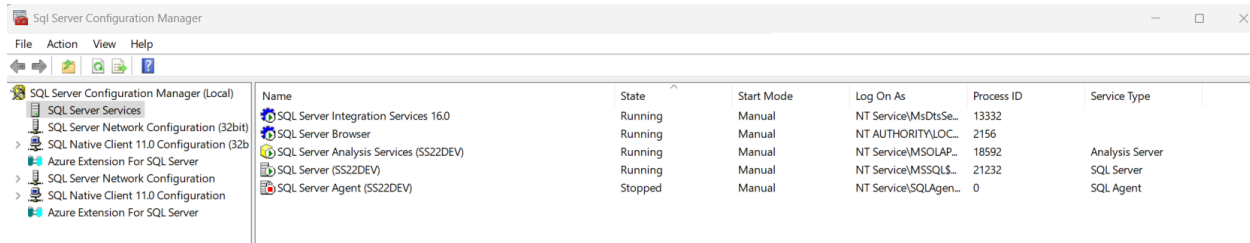


Fig 5.0: Configuration manager Settings

The SQL Server Import and Export Wizard was used to import the Access database (.mdb file) into SQL Server, using the Microsoft Access Database Engine as the source. A new SQL Server database was created to house the imported tables required for the BI project. The OLE DB SQL Provider served as the middleware for the ETL process, enabling smooth and accurate data transfer from Access to SQL Server. During this process, only the relevant tables and queries—such as dimStudent, dimGender, dimLocation, dimFeeStatus, dimDate, and the factFeePayments table—were selected to ensure a focused and efficient data warehouse.

Following the data import, Microsoft Visual Studio 2022 was used to create a new SQL Server Analysis Services (SSAS) project named "HostelManagement_1", using the Multidimensional and Data Mining Project template. Within the project, a data source (HostelManagement1DS.ds) and a data source view (HostelManagement1DSV.dsv) were created. These components were connected to the SQL Server database and allowed for visualizing the structure of dimension and fact tables. Logical primary keys were defined within the Data Source View, and relationships were drawn from the fact table to each dimension table, enabling multidimensional modeling. This setup laid the foundation for building an analytical cube for fee payment insights and further analysis through Excel or Power BI.

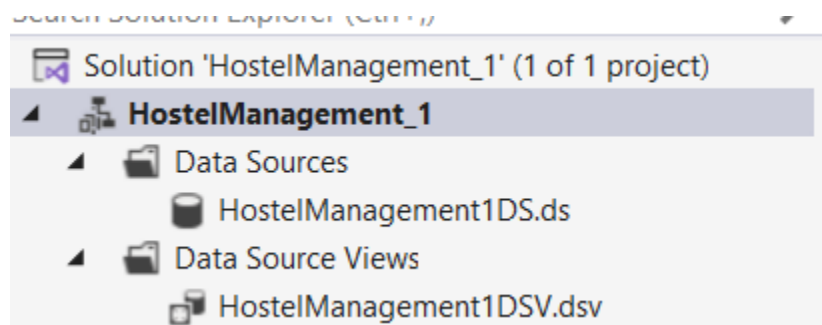


Fig 6.1: Connection of Data Source and Data Source View with the Database

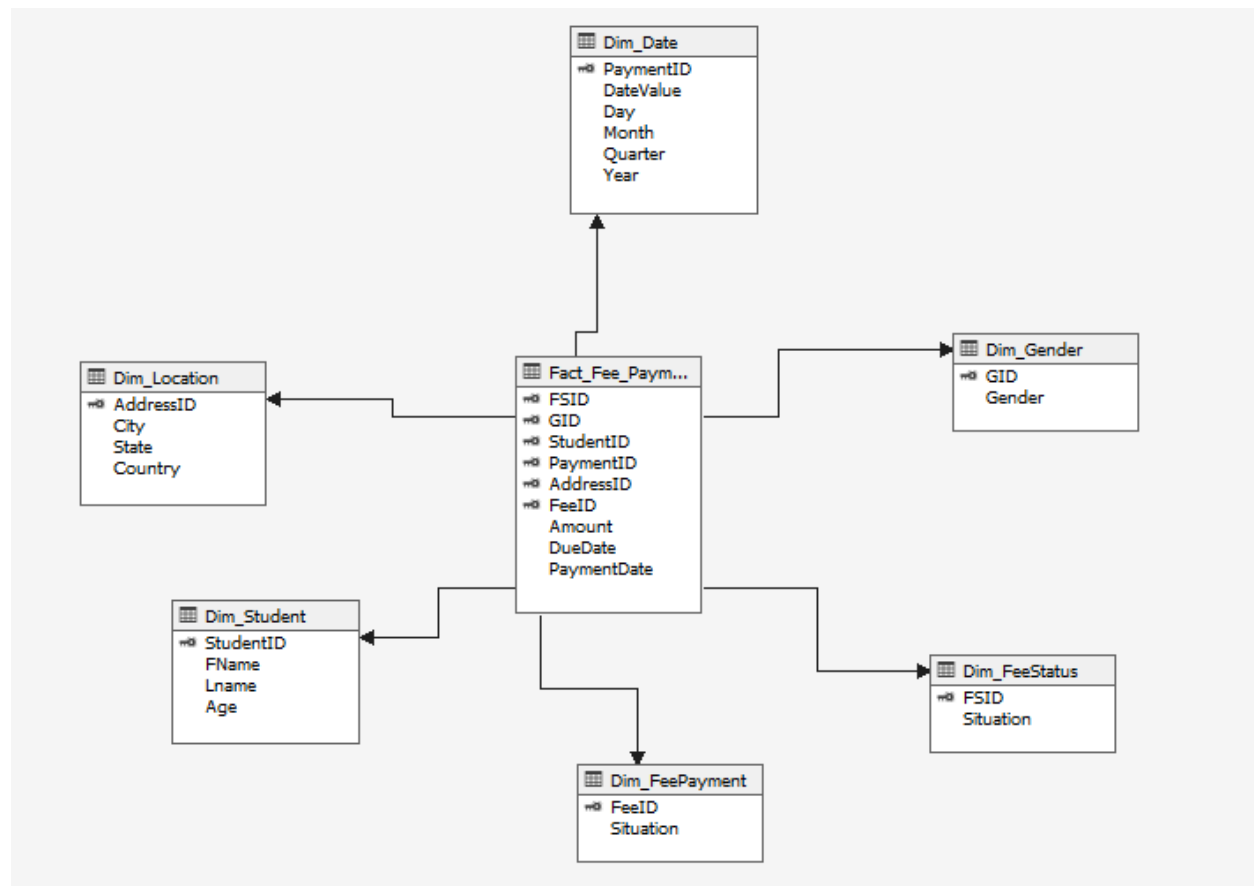


Fig 6.2: Data Source View (After setting logical relationships and primary key)

With the data source view properly configured, a cube was created using the available data. The resulting cube structure followed a perfect star schema, reflecting the relationships between the fact table and the various dimensions.

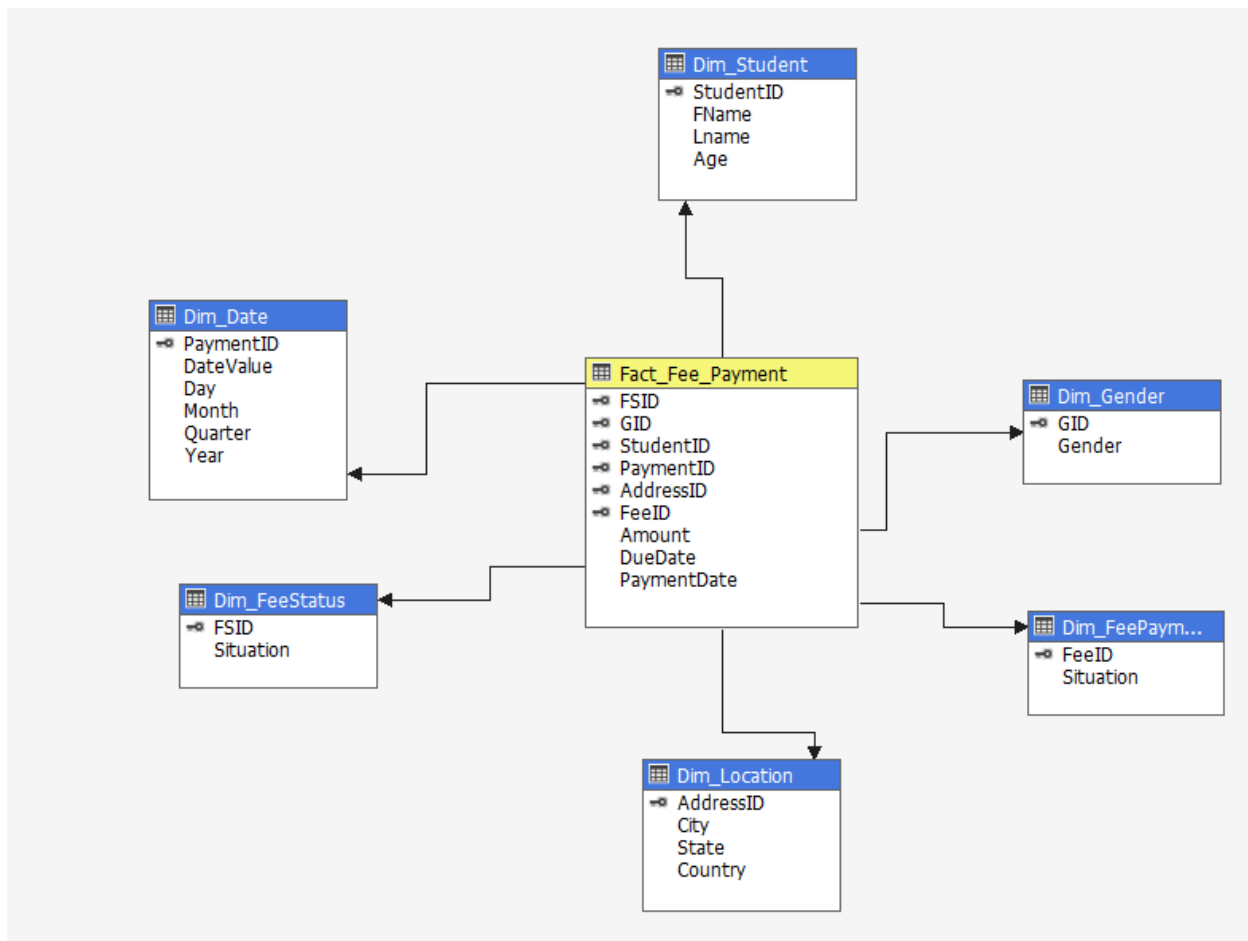


Fig 7.0: Cube Design

To enhance the usability and analytical capabilities of the Fee Payment Cube, hierarchies were defined within each dimension.

For example, in the Dim_Date dimension, a clear time-based hierarchy was created, starting from Year, followed by Quarter, then Month, and finally Day. This allowed users to drill down into fee payments across different time granularities for better trend analysis. In the Dim_Location dimension, the hierarchy was established as Country → State → City, which helps users analyze payment data regionally and understand geographic patterns in fee submissions. In the Dim_Student dimension, a basic hierarchy using First Name → Last Name was incorporated for identifying individual students, along with age-based segmentation for deeper insights into demographics.

Similarly, the Dim_FeeStatus and Dim_FeePayment dimensions contained fee situation levels, which were helpful in filtering the data by payment type (e.g., Full, Partial, Pending) and status (e.g., Paid, Unpaid).

Lastly, the Dim_Gender dimension, though flat, allowed for gender-wise segmentation of students, which can be used for reporting and equity analysis. These hierarchies significantly improved the

user experience during cube exploration in tools like Excel or Power BI, enabling effective slicing, dicing, and filtering of data with minimal effort.

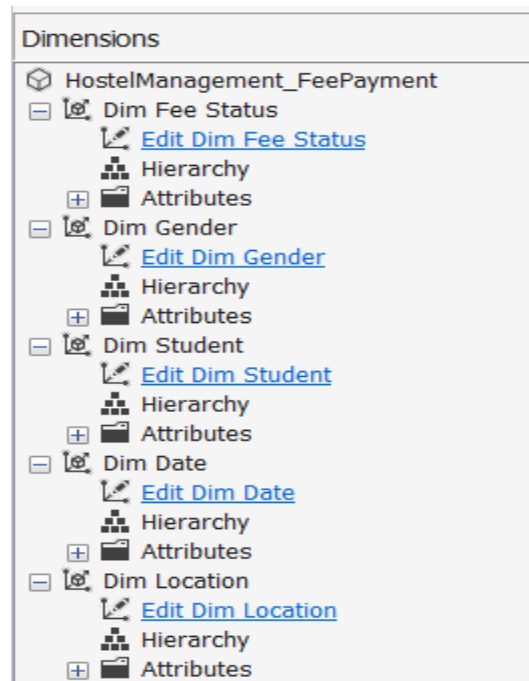


Fig 8.1: Edit Dimensions and Creating Hierarchy

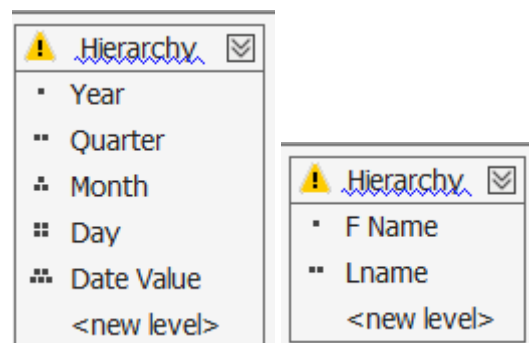


Fig 8.2: Hierarchies design for dimdate and dimstudent



Fig 8.3: Hierarchies design for dimlocation

Before processing the cube, two crucial settings were adjusted. First, in the deployment section of the data source properties, the localhost was changed to the laptop name (Charmy\SS22DEV) to ensure proper connectivity.

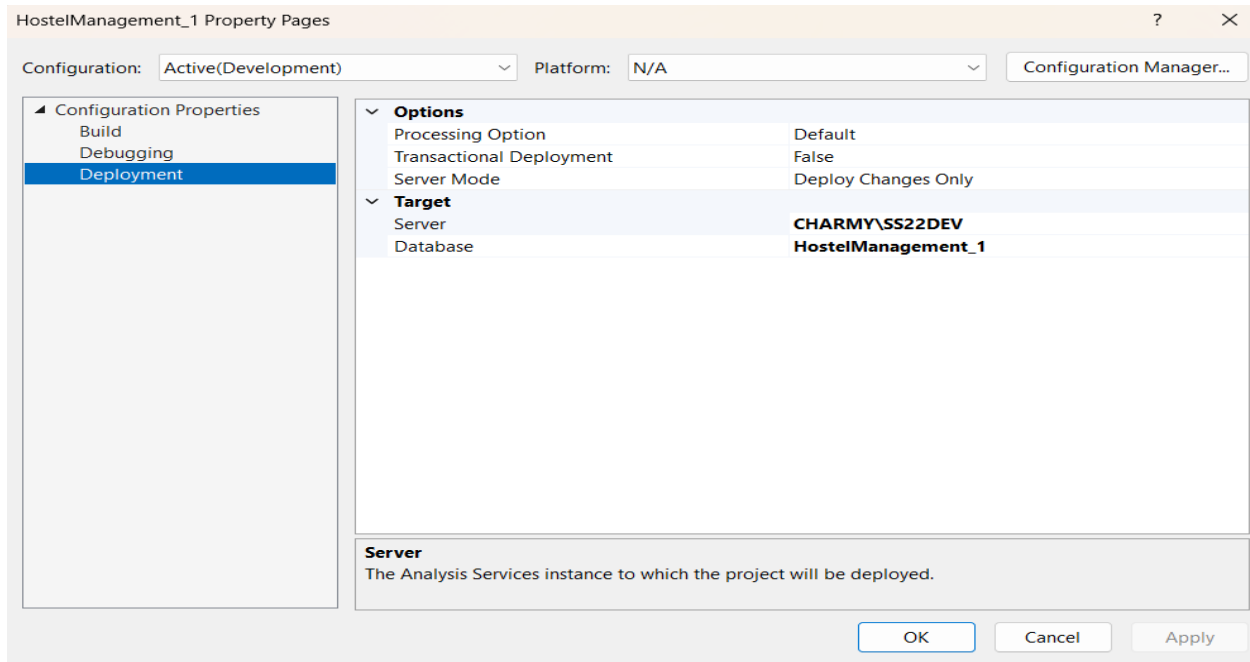


Fig 9.0: Data Source Properties Setting Changed (server setting in deployment)

Second, the appropriate username and password for the laptop were provided to establish a secure connection. With these settings in place, the cube was processed successfully.

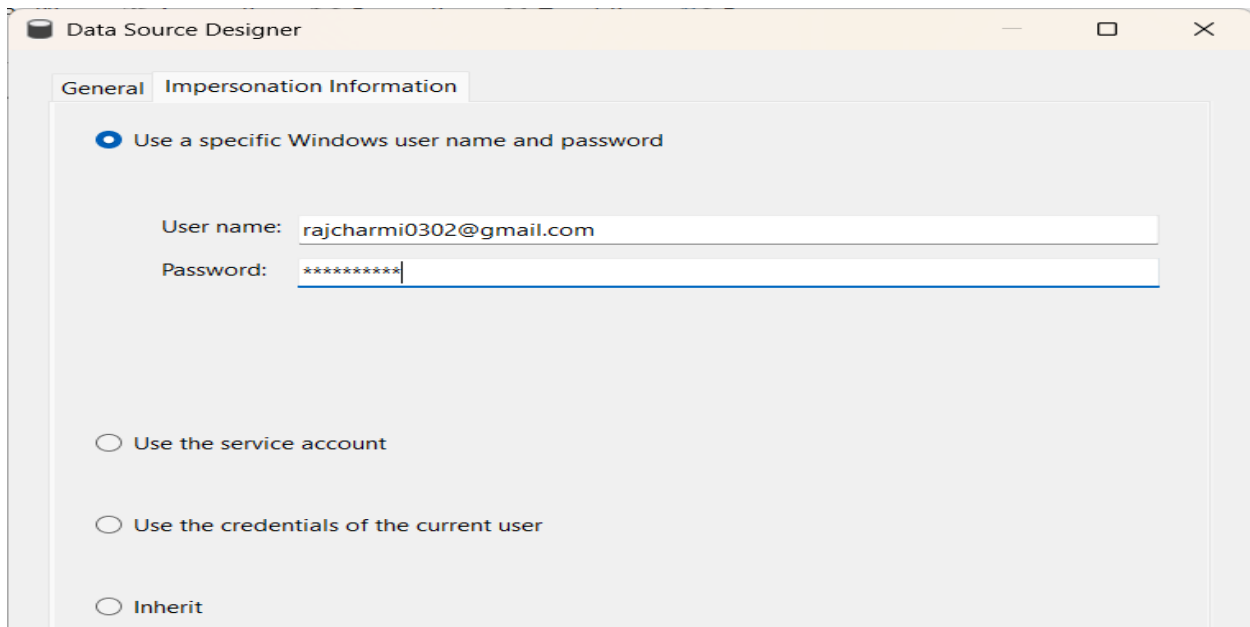


Fig 10.0: Input password and username in Impersonation Information

Upon running the cube, a "Process Succeeded" message was received, indicating that the cube was ready for analysis. The project was then saved, and the cube was browsed to verify its functionality.

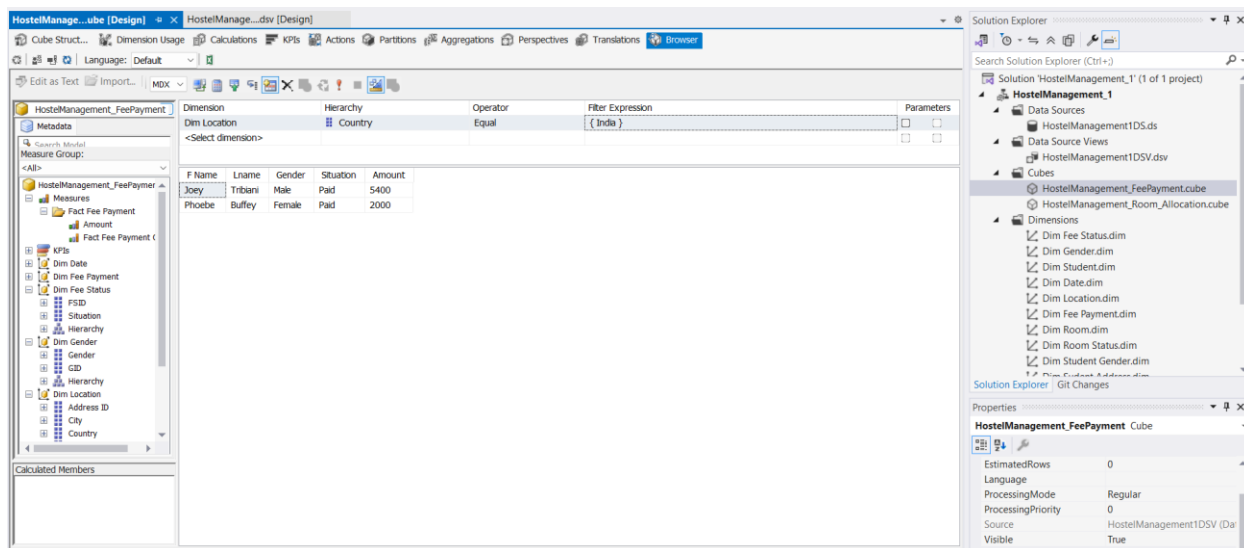


Fig 11.0: Successfully Browsing the cube with different Facts and Dimensions

To enable offline access to the cube and facilitate data sharing, Microsoft Excel was utilized. By connecting to the Analysis Services database and selecting the appropriate data source file, a new online cube was created using the OLAP tool in Excel. This process generated a .cub file containing pivot tables, pivot report fields, and charts, all of which were tailored to meet the specific business justification and reporting requirements of Hostel Management System. The resulting reports delivered clear, actionable insights into student fee payments, including due amounts, payment status, payment trends over time, gender-based payment patterns, and regional payment breakdowns. These reports allowed the hostel administration to make data-driven decisions, such as identifying defaulters, forecasting collections, monitoring late payments, and understanding fee trends across different locations and student demographics.

This approach not only streamlined financial reporting but also empowered stakeholders with easily accessible, offline, and interactive reports, improving both operational efficiency and strategic planning.

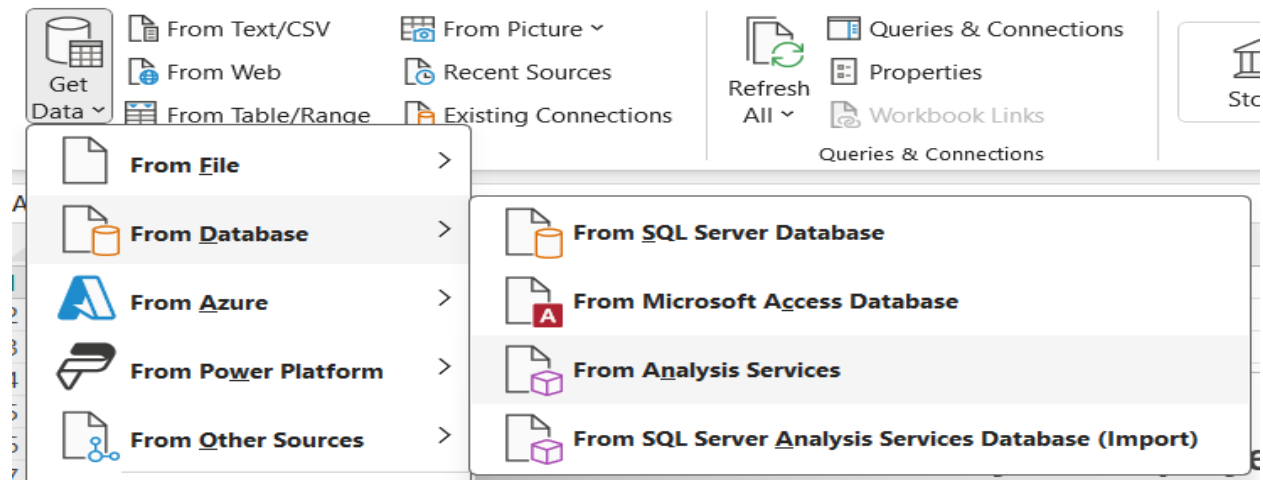


Fig 12.1: Import Data in excel via Analysis Services

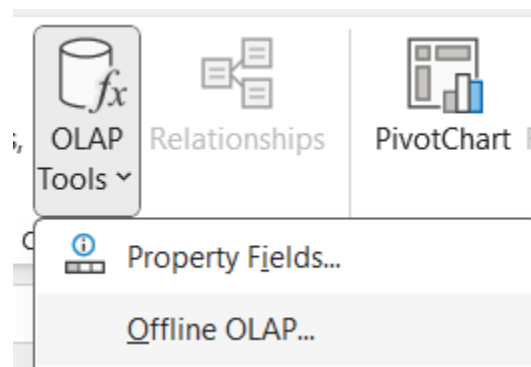


Fig 12.2: Create Offline Cube with OLAP Tools

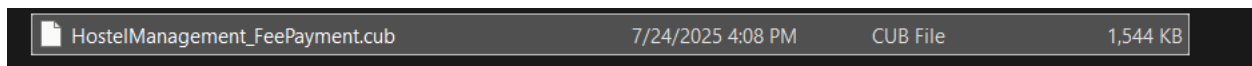


Fig 12.3: .CUB file created.

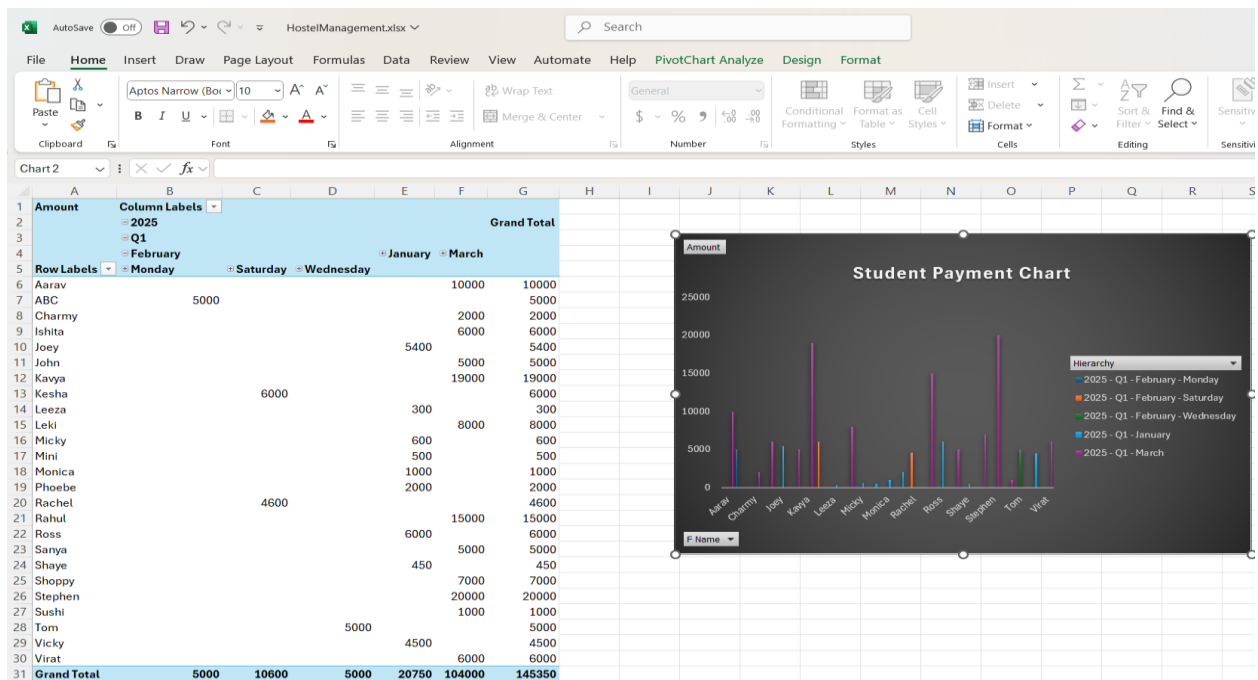


Fig 12.4: Offline cube with pivot table fields and pivot report fields & created multiple reports in different excel sheets.

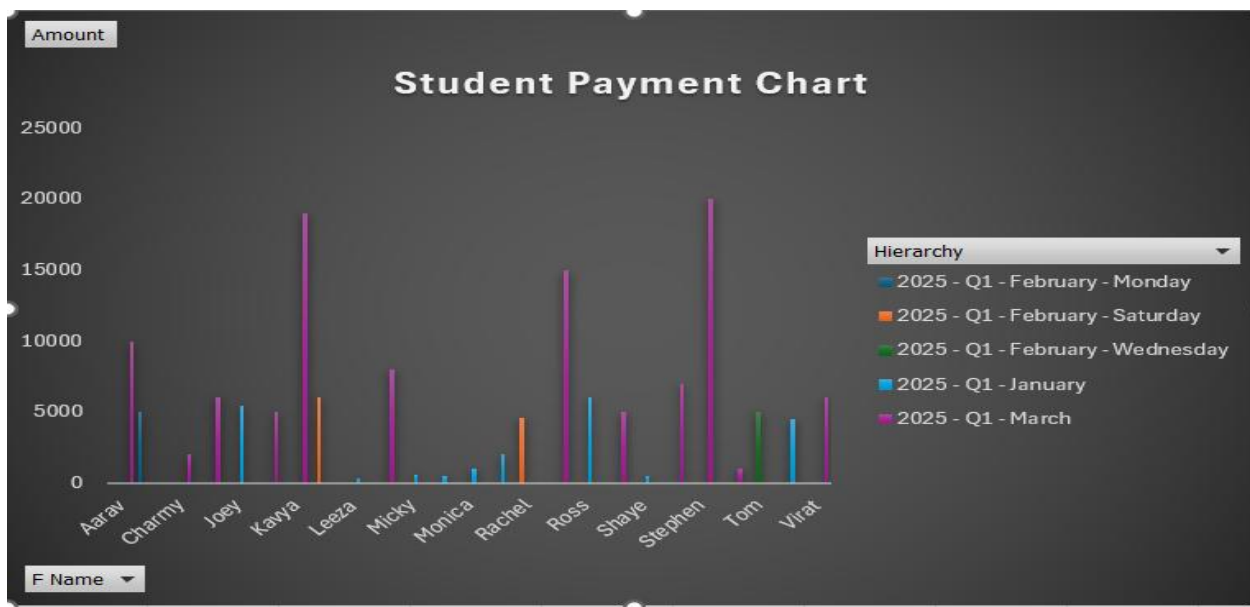


Fig 12.5: Student Payment Chart

Cube 2: Room Allocation Cube

To enable offline access to the cube and facilitate data sharing, Microsoft Excel was utilized. By connecting to the Analysis Services database and selecting the appropriate data source file, a new online cube was created using the OLAP PivotTable feature in Excel. This process generated a .cub file containing pivot tables, report filters, slicers, and charts, specifically customized to reflect the analytics required for Hostel Room Allocation and Occupancy Cube.

The generated reports offered critical insights into room allocation patterns, such as occupancy rates by gender, age group, location, and time period. Additionally, the cube enabled analysis of room availability, status of room assignments, and student demographic trends over different academic terms. The data visualization allowed the hostel management to identify underutilized rooms, plan for peak occupancy periods, and optimize room distribution strategies.

By using this offline cube solution, the administrative team was able to access and analyze data interactively, without relying on a constant database connection, enhancing decision-making and operational planning across the hostel facilities.

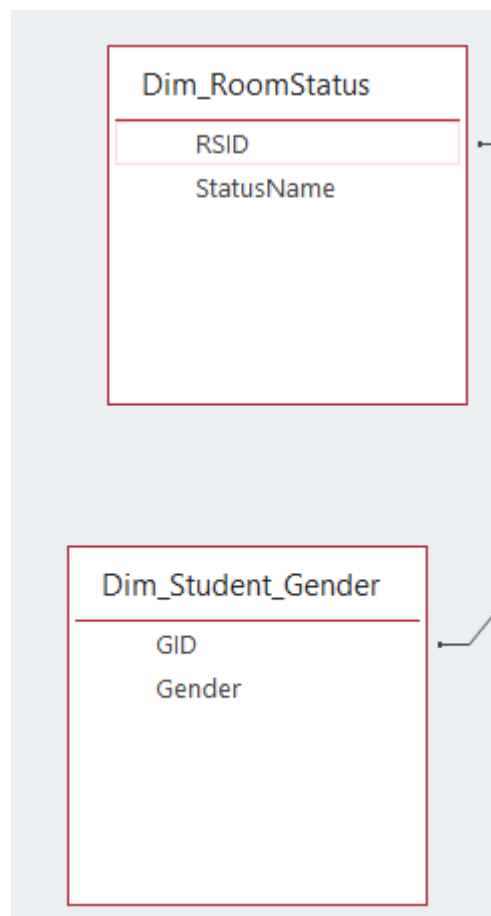


Fig 13.1: Dimension RoomStatus and Student_Gender Query Field

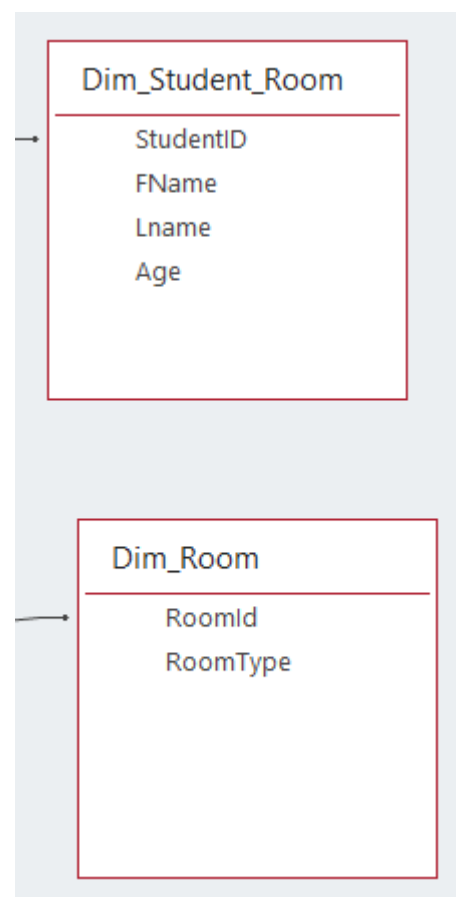


Fig 13.2: Dimension Student_Room & Room

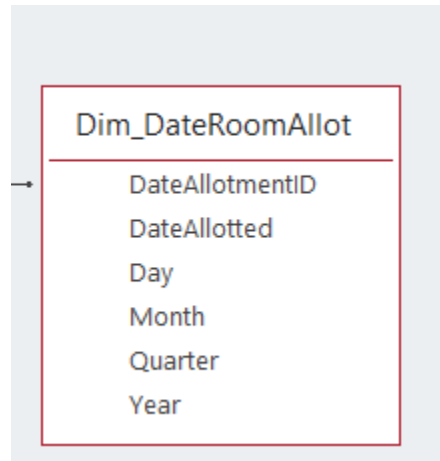


Fig 13.3: Dimension DateRoomAllot Query Field

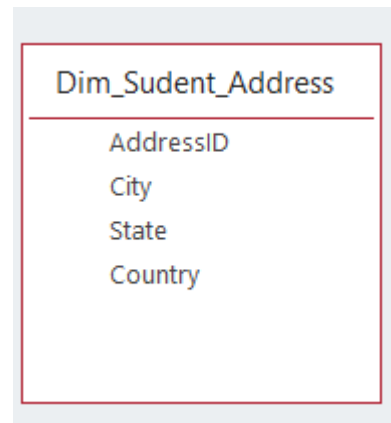


Fig 13.4: Dimension Student_Address Query Field

To handle the time dimension in the Room Allocation and Occupancy cube, a separate table named AllotmentDate was created in Excel and then imported into the Access database. This table included detailed date-related attributes such as Day, Month, Quarter, and Year.

The AllotmentDate table was then connected to the Fact_Room_Allocation table using the DateRoomAllot attribute. This connection ensured that the time dimension was properly integrated with the fact data, allowing for accurate time-series analysis of room usage trends.

Importantly, this setup avoided the risk of creating a cross-cartesian product, which could have led to data duplication or inaccurate reporting. With the time dimension structured separately, users could analyze room occupancy patterns over various time periods; daily, monthly, quarterly, or yearly- with ease and consistency.

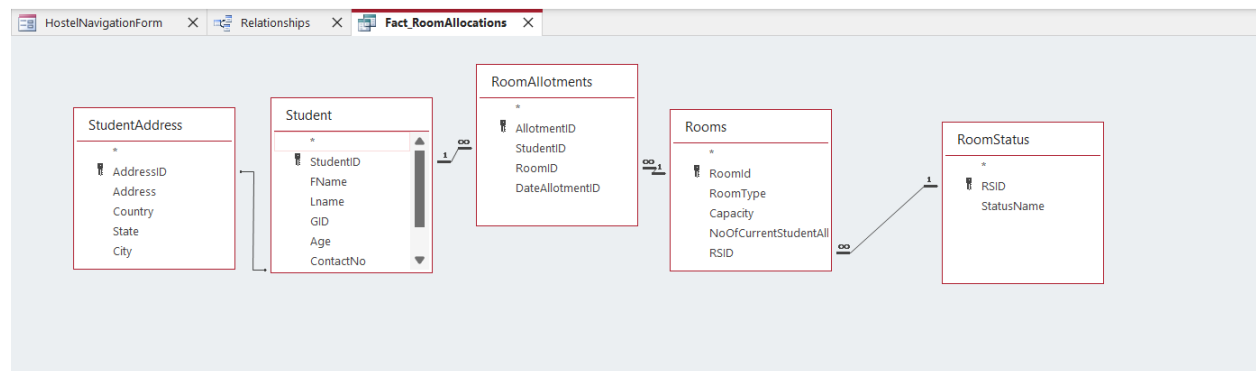


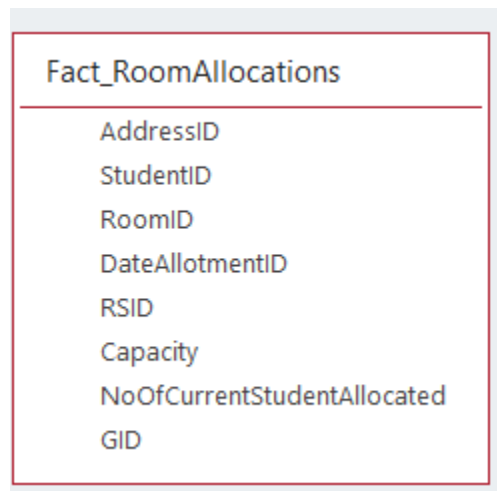
Fig 14.0: Create Time relationship in Fact query.

The fact table query for Cube 2 was designed to include all the essential primary keys; StudentID, RoomID, DateAllotmentID, AddressID, RSID, and GID, to establish robust links with the relevant dimension tables. The key measures included were Capacity and NoOfCurrentStudentAllocated, which collectively provide detailed insights into room usage.

This setup was carefully structured to fulfill the main objective of the project: enabling meaningful reporting on student accommodation trends. By tracking which students were allotted to which rooms, when the allocation occurred, the gender distribution, and the status of each room (e.g., Occupied or Available), the cube delivers powerful insights into:

- Hostel occupancy rates
- Student movement patterns
- Capacity utilization
- Gender-based room distribution

Including these specific keys and measures enhanced the hostel’s ability to make data-driven decisions related to space management, optimize room allocation, and proactively address issues like overbooking or underutilization. Ultimately, the cube plays a critical role in improving operational efficiency and ensuring that room allocation is handled fairly and effectively across the hostel system.

The image shows a screenshot of a database table named 'Fact_RoomAllocations'. The table is displayed within a window that has a light blue header bar. The table itself has a white background with a thin red border. The fields are listed vertically, and each field name is preceded by a small blue icon. The fields are: AddressID, StudentID, RoomID, DateAllotmentID, RSID, Capacity, NoOfCurrentStudentAllocated, and GID. To the right of the table, there are three small blue icons, each with a horizontal line extending to the right, indicating a relationship or link to another table.

Fact_RoomAllocations	
AddressID	
StudentID	
RoomID	
DateAllotmentID	
RSID	
Capacity	
NoOfCurrentStudentAllocated	
GID	

Fig 15.0: Fact Query Field

After creating the dimension and fact table queries, relationships were established between the tables using the relationship tab in Microsoft Access. The dimStudent, dimRoom, dimDateAllotment, dimRoomStatus, dimLocation, and dimGender tables were all connected to the Fact_RoomAllocations table, ensuring data integrity and enabling accurate analysis.

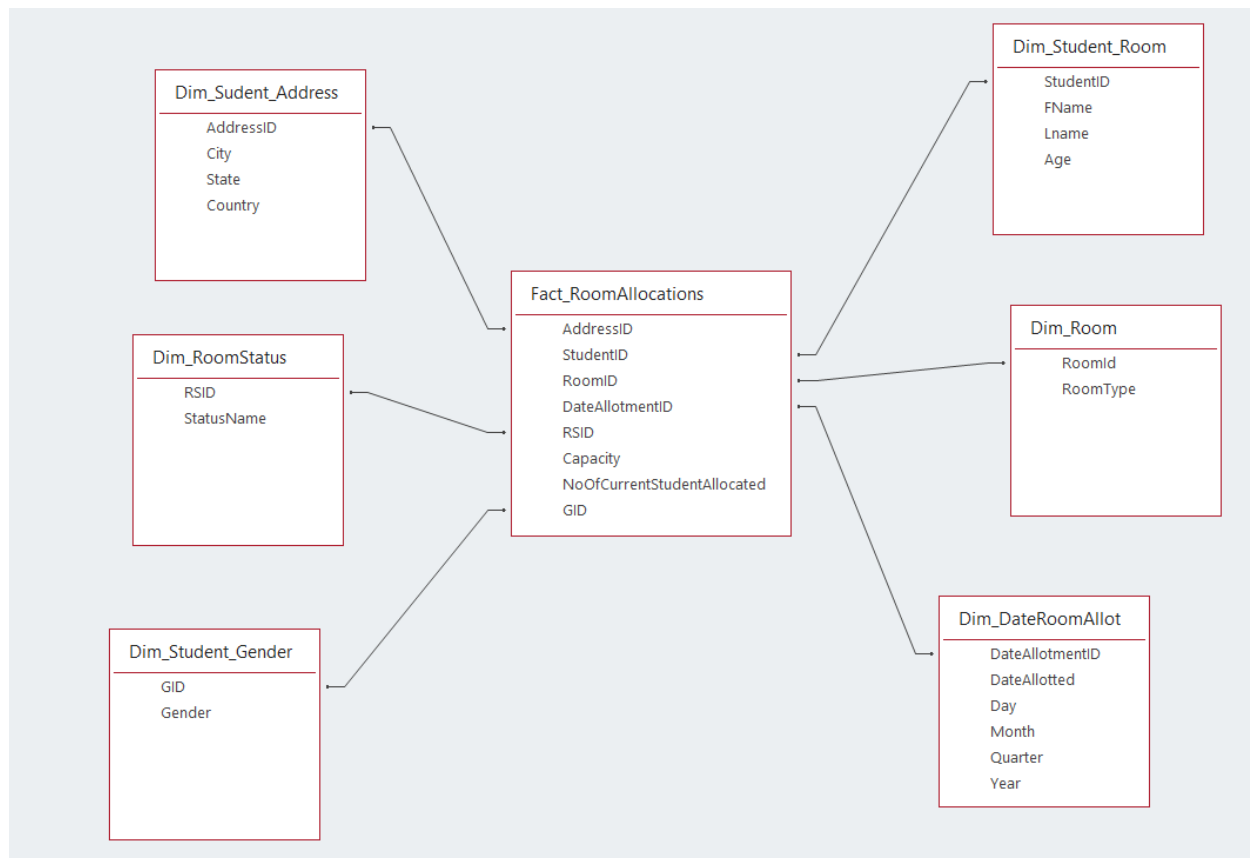


Fig 16.0: Established Relationship with Fact Table

With the Access database prepared, the next step involved uploading the data into Microsoft SQL Server 2022. To accomplish this, the SQL Server Configuration Manager was used to start the necessary services, including SQL Server Integration Services, SQL Server Analysis Services, SQL Server, and SQL Server Browser.

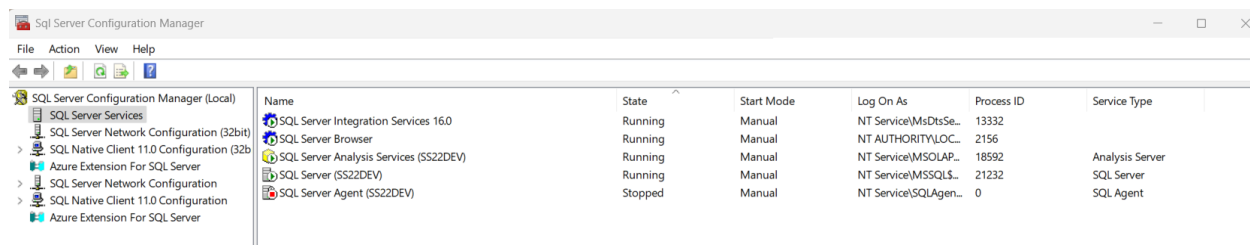


Fig 17.0: Configuration manager Settings

The SQL Server Import and Export Wizard was then employed to import the Access database (.mdb file) into SQL Server, using the Microsoft Access Database Engine as the source and creating a new database named "HostelManagementDW." The OLE DB SQL Provider was utilized as the middleware for the ETL process, facilitating the seamless transfer of data from Access to SQL Server. During the import process, the specific tables and queries required for the BI system

(dimStudent, dimRoom, dimDateAllotment, dimRoomStatus, dimLocation, dimGender, and Fact_RoomAllocations) were selected, ensuring a clean and focused data warehouse.

Following the successful data import into SQL Server, Microsoft Visual Studio 2022 was used to create a new repository for the data mart. A new Analysis Services project with a multidimensional template was initiated, resulting in the creation of the "HostelManagement_1" repository. Within Visual Studio, the data source (HostelManagement1DS.ds) and data source view (HostelManagement1DSV.dsv) were connected, allowing for the visualization of the dimension and fact tables in the design tab. Logical primary keys were set for each table, and relationships were established by connecting the primary keys from the fact table to their corresponding dimension tables, laying the groundwork for cube creation.

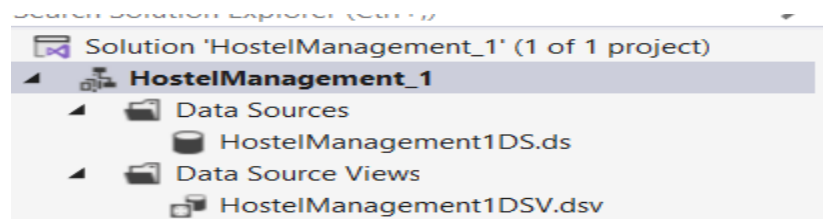


Fig 18.1: Connection of Data Source and Data Source View with the Database

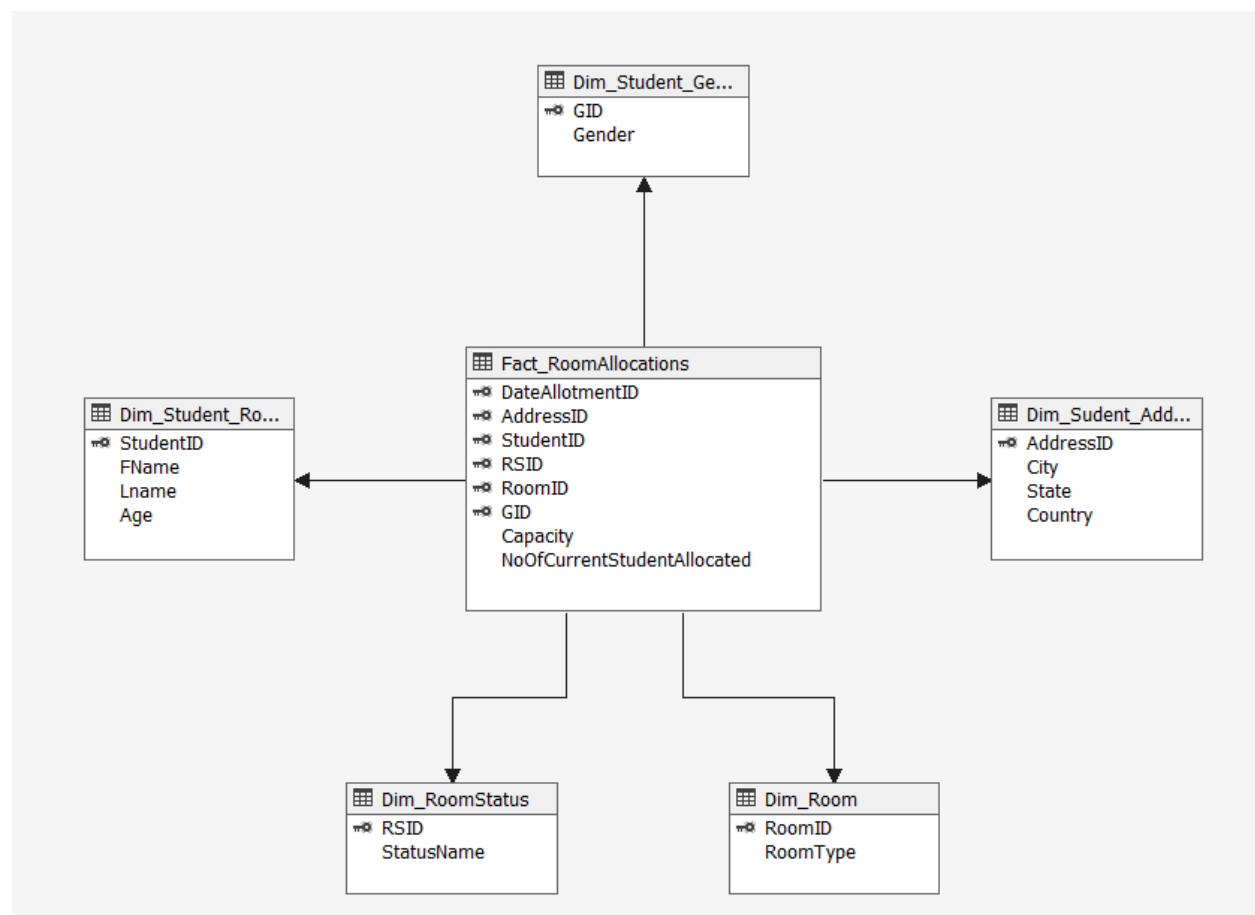


Fig 18.2: Data Source View (After setting logical relationships and primary key)

With the data source view properly configured, a cube was created using the available data. The resulting cube structure followed a perfect star schema, reflecting the relationships between the fact table and the various dimensions.

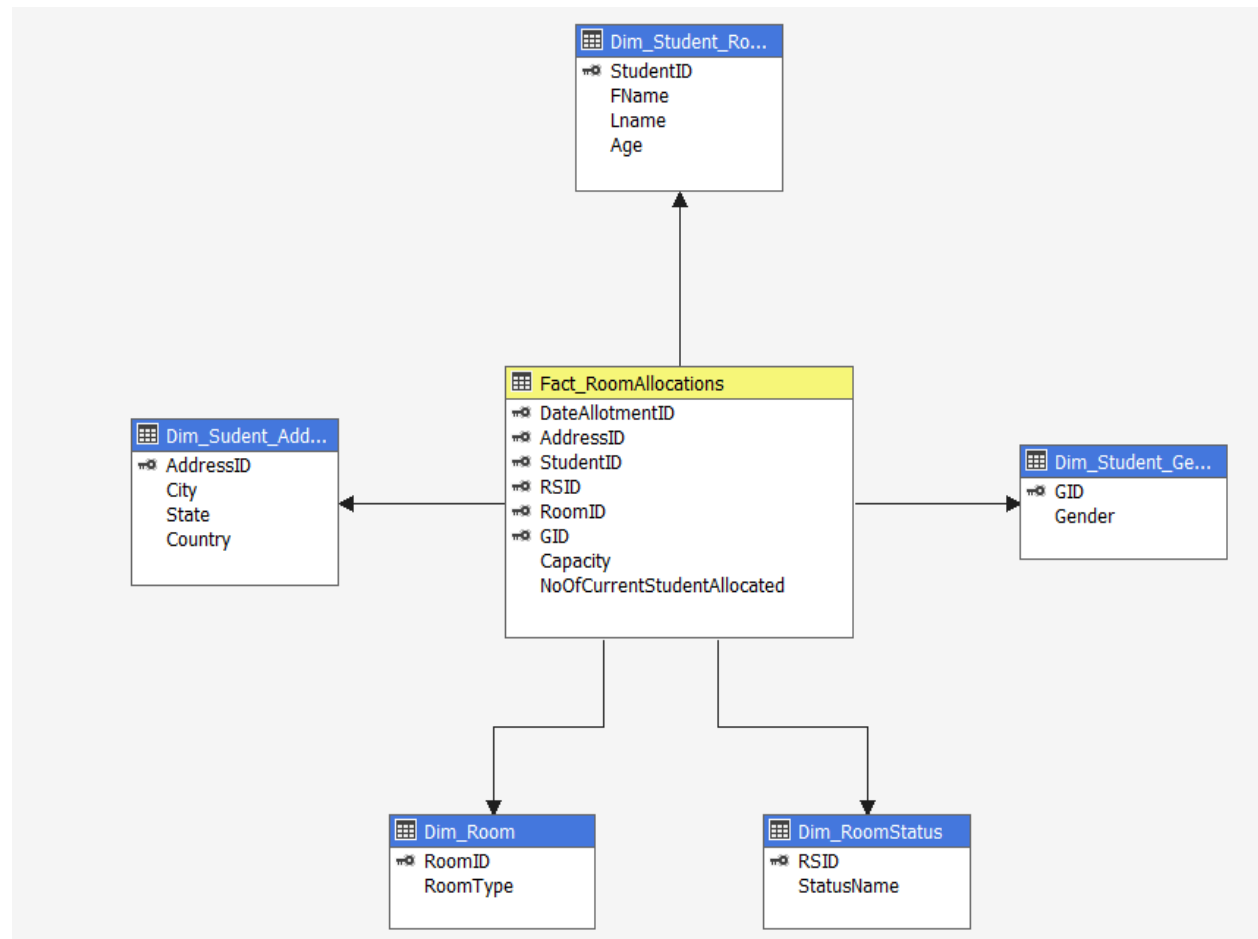


Fig 19.0: Cube Design

To enhance the usability and analytical capabilities of the cube, hierarchies were defined within each dimension. For example, in the dimStudent dimension, the first name (FName) was set as the first level of the hierarchy, followed by the last name (LName) and then the age. Similar hierarchies were created for the dimRoomStatus, dimDateAllotment, dimLocation, and dimGender dimensions, using their respective attributes.

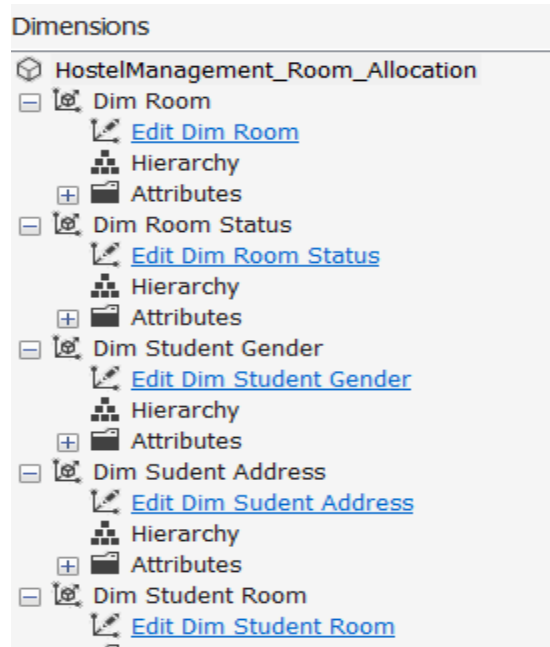


Fig 20.1: Edit Dimensions and Creating Hierarchy

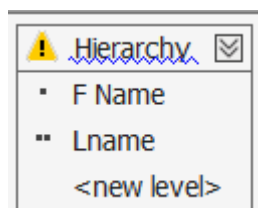


Fig 20.2: Hierarchies design for Dim_Student_Room



Fig 20.3: Hierarchies design for Dim_Student_Address

Before processing the cube, two crucial settings were adjusted. First, in the deployment section of the data source properties, the localhost was changed to the laptop name (Charmy\SS22DEV) to ensure proper connectivity.

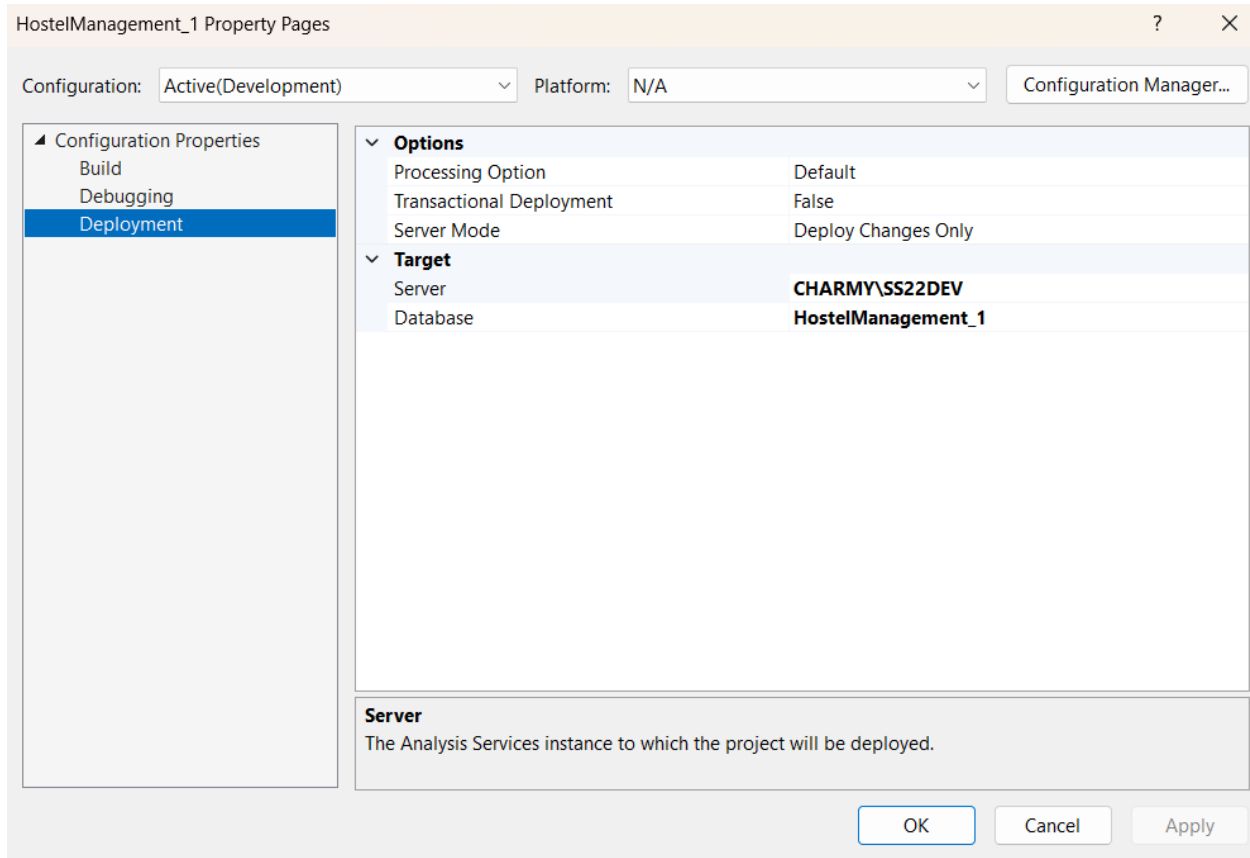


Fig 21.0: Data Source Properties Setting Changed (server setting in deployment)

Second, the appropriate username and password for the laptop were provided to establish a secure connection. With these settings in place, the cube was processed successfully.

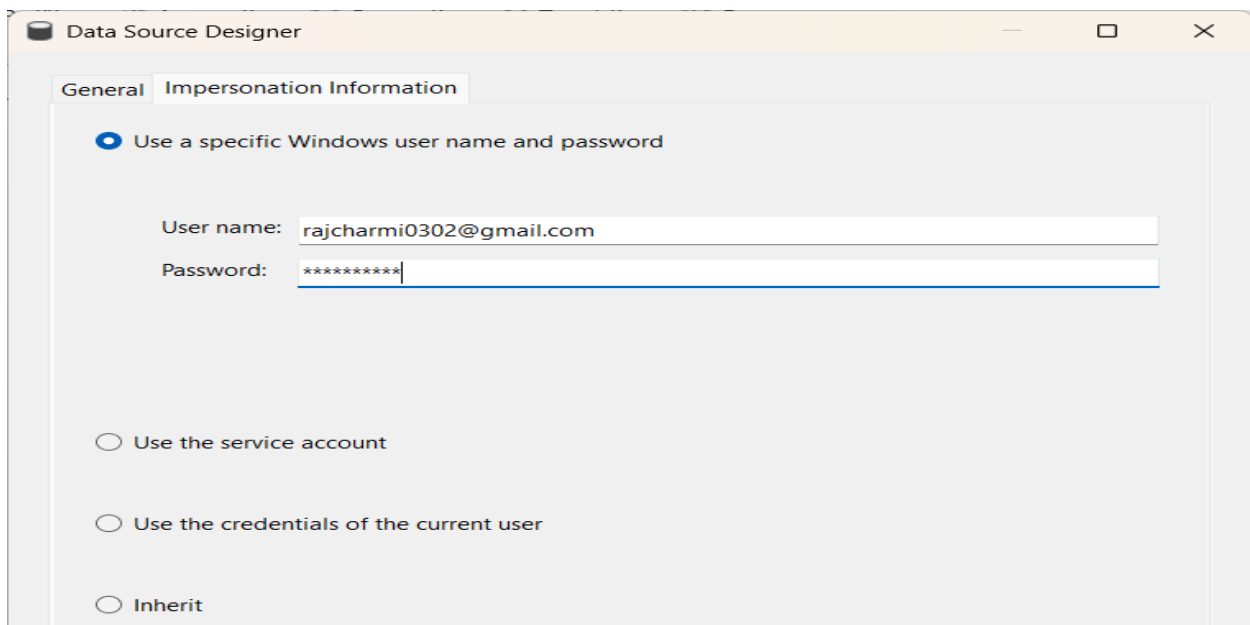


Fig 22.0: Input password and username in Impersonation Information

Upon running the cube, a "Process Succeeded" message was received, indicating that the cube was ready for analysis. The project was then saved, and the cube was browsed to verify its functionality.

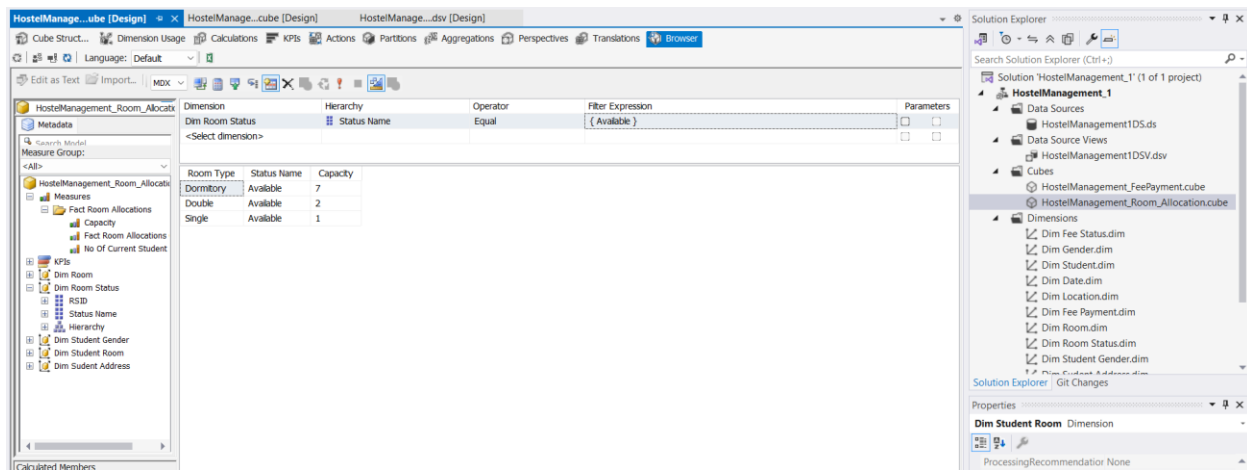


Fig 23.0: Successfully Browsing the cube with different Facts and Dimensions

To enable offline access to the cube and facilitate data sharing, Microsoft Excel was utilized. By connecting to the Analysis Services database and selecting the appropriate data source file, a new online cube was created using the OLAP tool in Excel. This process generated a .cub file containing pivot tables, pivot report fields, and charts, all of which were tailored to meet the specific business justification and reporting requirements of Hostel Management System. The resulting reports provided valuable insights into student accommodation trends, room allocation status, gender distribution, and hostel capacity utilization, empowering the institution to make informed decisions based on comprehensive and easily accessible data.

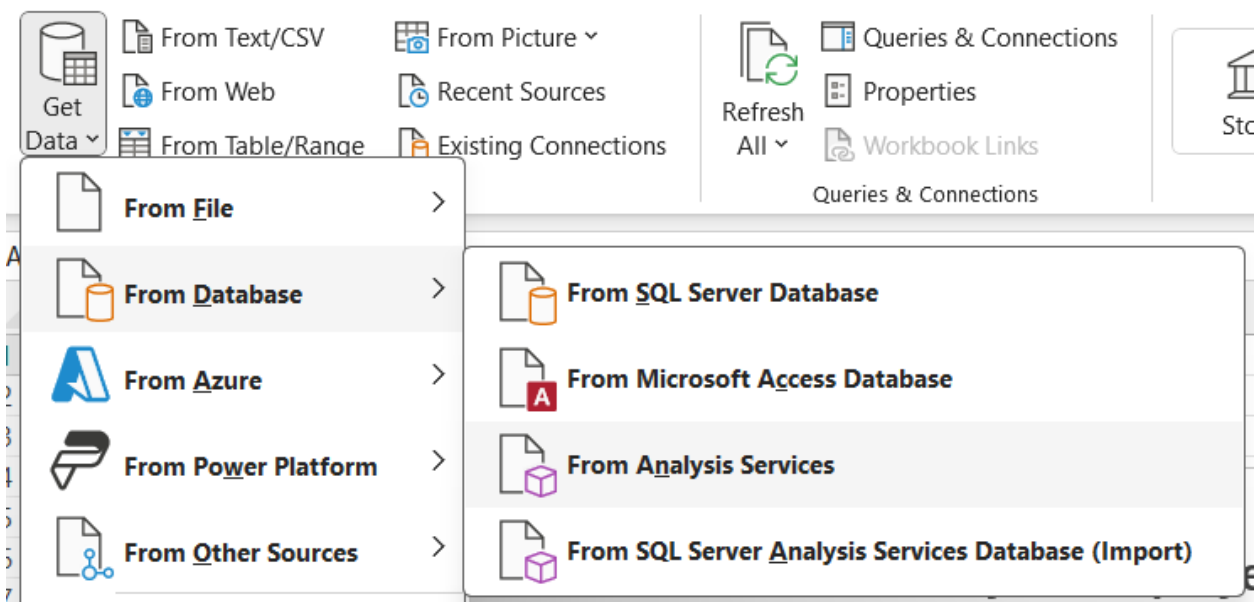


Fig 24.1: Import Data in excel via Analysis Services

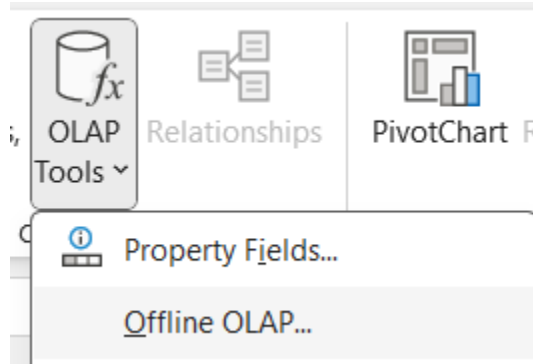


Fig 24.2: Create Offline Cube with OLAP Tools

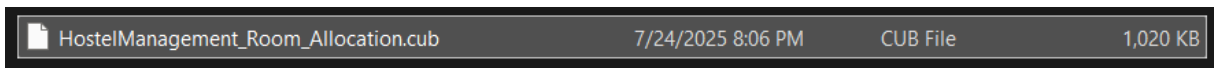


Fig 25.3: .CUB file created.

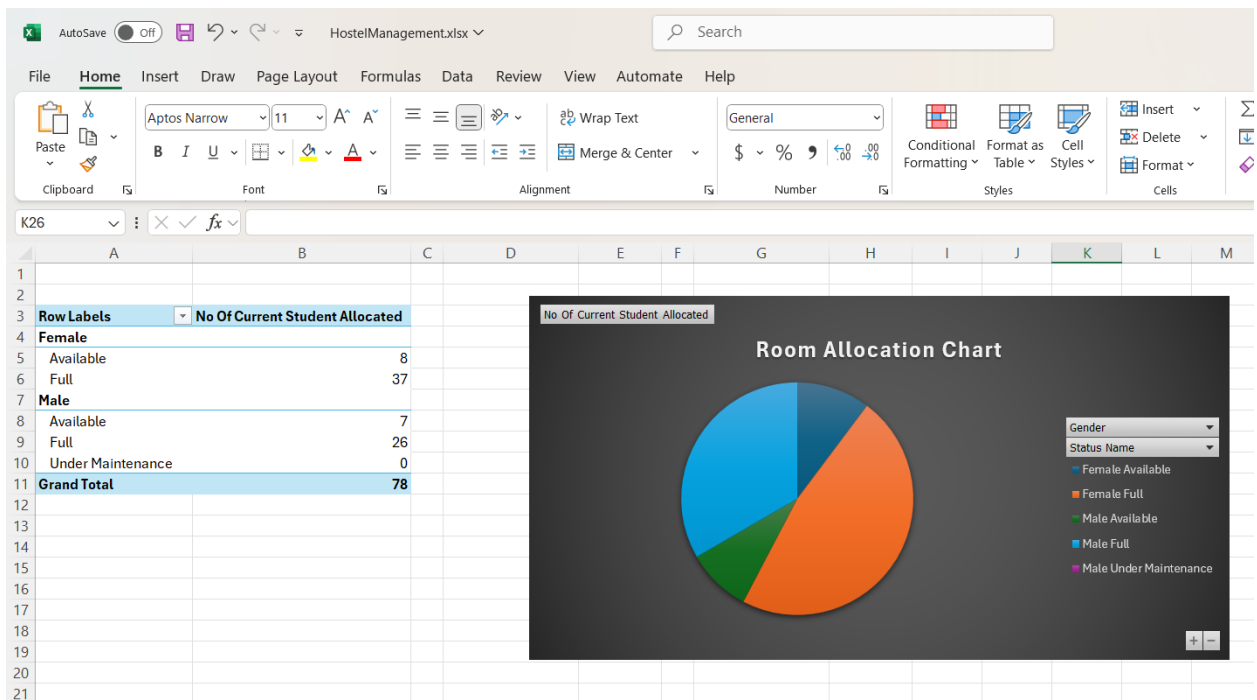


Fig 24.4: Offline cube with pivot table fields and pivot report fields & created multiple reports in different excel sheets.

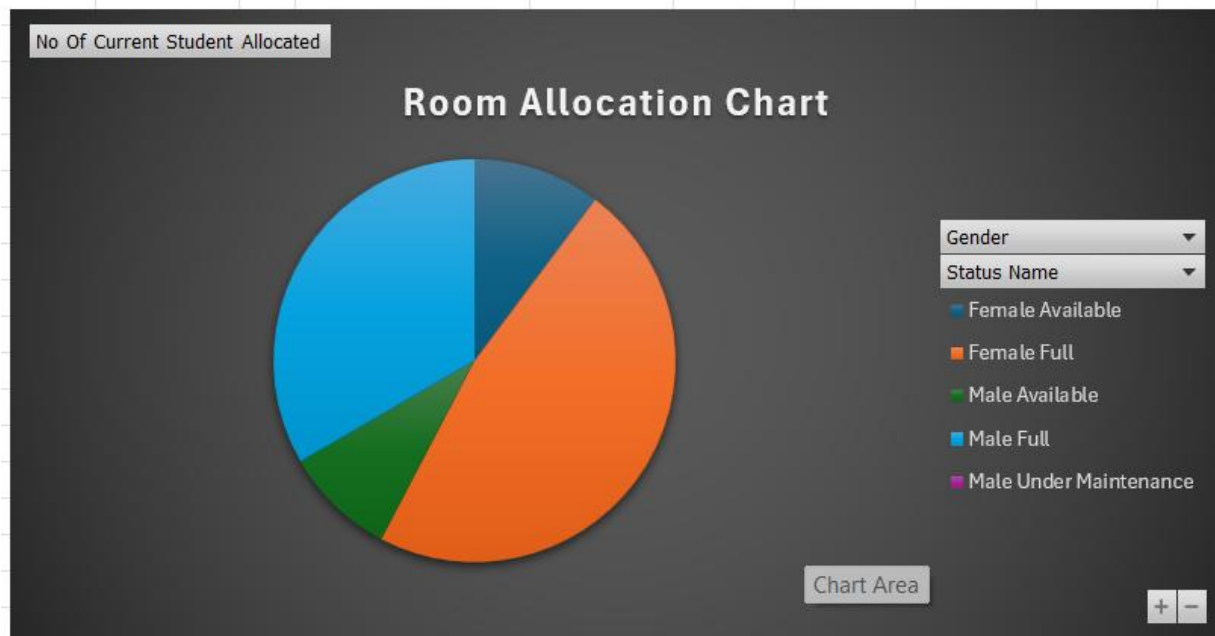


Fig 24.5: Example of reports (Travel itinerary & cost report)

Lessons Learned/3Ws:

a. What went well:

Throughout this project, I experienced several wins that gave me confidence in my ability to apply Business Intelligence techniques effectively. Some of the key things that went well:

- **Star Schema Design:**
I successfully designed normalized **star schemas** for both cubes, ensuring that dimension tables were cleanly separated from the fact tables. This helped maintain clarity in relationships and simplified the cube design process.
- **SSAS Cube Deployment:**
Both the **Fee Payment Cube** and **Room Allocation Cube** were deployed and processed **without any structural errors** in SQL Server Analysis Services (SSAS). This validated that my schema, key mappings, and data types were well aligned.
- **Pivot Table Analysis in Excel:**
I was able to connect Excel to my cubes via **OLAP Pivot Tables**, making it easy to filter and visualize data interactively. This was especially helpful in demonstrating insights to others who may not be familiar with SQL.

- **Improved Conceptual Understanding:**
One of the most important outcomes was my improved understanding of **multidimensional data modeling, fact-dimension relationships**, and how to think analytically from a BI perspective. I now feel more confident working with cubes, hierarchies, and measures.

b. What did not go well:

While the overall project was a success, I encountered several technical and conceptual challenges that slowed down progress and tested my problem-solving abilities. These are important to acknowledge, as they offered key learning moments:

1. Data Type Mapping Issues Between Access and SSAS

When I imported data from Microsoft Access into SQL Server Analysis Services (SSAS), I frequently encountered **data type mismatches**—especially with fields like PaymentDate, ContactNo, and IDs. Some numeric fields were interpreted as text, while date fields came in with inconsistent formats. These mismatches caused errors during cube processing and dimension linking. I had to go back and **manually clean or cast** the data in SQL multiple times, which was time-consuming. I realized that being casual about data types early on can lead to a lot of frustration later in the process.

2. Difficulties Identifying Students Who Did Not Pay Fees

A major insight I wanted to extract was "**Which students haven't paid their fees yet?**", but I struggled to implement this. Since students who didn't pay fees had no corresponding PaymentDate, those records simply didn't exist in the fact table (Fact_FeePayments). This exposed a limitation of **multidimensional cubes**, which are designed to summarize **existing data**, not highlight **missing data**. In the future, I would consider creating a **default "Not Paid" entry** or use **outer joins during data staging** to make sure these students are represented in the cube with a clear fee status.

3. Cross-Join Issues When Connecting Room Dimension

While building the **Room Allocation Cube**, I initially faced **cartesian join issues** when connecting Dim_Room and Fact_RoomAllocations. Improper or missing foreign keys caused SSAS to multiply rows inappropriately, leading to **incorrect aggregates and duplicate data** in the cube. It took multiple iterations to identify the exact cause; I had overlooked enforcing referential integrity and indexing Room IDs properly in both tables. This highlighted the importance of carefully validating **primary/foreign key consistency** before cube design.

4. ETL Workflow Repetition

Every time I made small schema changes (like renaming a column or changing a type), I had to **recreate the entire ETL pipeline manually**; from Access export to SQL import to SSAS reprocessing. This was tiring and repetitive, and it often introduced new errors. In hindsight, I should have **scripted or automated the ETL process** using SQL Server Integration

Services (SSIS) or reusable SQL scripts. This would've saved me a lot of time and reduced human error.

5. Lack of Error Logging or Warnings in Access

Microsoft Access didn't always provide clear error messages or warnings. Sometimes records would fail to export or process in SSAS without obvious explanations. Debugging felt like a black box at times, and I had to manually cross-check thousands of records to find inconsistencies. In the future, I would consider using a more robust ETL tool with **logging, error handling, and data validation features**.

c. What would you do differently next time:

This project was a huge learning experience, and it gave me several takeaways that I would carry forward into my next data warehousing or BI project:

- **Plan Data Relationships in Advance:**
I would spend more time **visually mapping entity relationships and keys before loading into SSAS**. This would help avoid relationship errors and reduce cube processing failures.
- **Avoid Manual ETL Repetition:**
Every time I made a change to the database; I had to manually redo the entire ETL process. Next time, I will **invest in reusable ETL templates or automation scripts** (using SSIS or stored procedures) to save time and minimize rework.
- **Design More Cubes for Broader BI:**
This project made me realize the power of cubes for different hostel operations. In future projects, I'd like to build additional cubes; such as:
 - **Mess Management Cube**
 - **Student Complaint & Feedback Cube**
 - **Maintenance Requests Cube**
- **Handle Missing Data Strategically:**
I now understand the importance of **handling NULLs** and missing data at the source level. Next time, I'll create a dedicated dimension value like 'No Payment' or 'Not Assigned' to allow such cases to be tracked properly inside the cube.

Conclusion:

This project has been a valuable and rewarding journey that allowed me to explore the real-world application of Business Intelligence tools to solve operational problems within a hostel management context. From data collection and schema design to cube creation and interactive visual analysis, each stage deepened my understanding of how structured data can drive meaningful insights.

By designing two key data marts; **Fee Payment** and **Room Allocation**, I was able to build star schemas, deploy cubes in SQL Server Analysis Services (SSAS), and visualize data using Excel pivot tables. These tools enabled actionable insights, such as identifying defaulters, understanding room occupancy trends, and assessing hostel resource allocation.

Despite facing some technical hurdles, such as data type conflicts and issues with handling missing values, these challenges ultimately improved my problem-solving skills and understanding of ETL, multidimensional modeling, and OLAP systems.

Most importantly, this project has shown me how scalable and insightful BI solutions can be when implemented with care. It has inspired me to continue improving this system, possibly integrating it with **Power BI dashboards**, **automated alerts**, and expanding it with new cubes for mess, feedback, or maintenance tracking.

Overall, this project not only enhanced my technical capabilities but also gave me a clearer perspective on the strategic value of data in improving student experience and operational efficiency.